

AADK 6 - Task 1 — Button Interaction

Fundamentals & Analysis

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 6

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

Sample Compose Code Snippet (Button with Click Counter)

@Composable

```
fun ClickCounter() {
```

```
    // (1) STATE DEFINITION
```

```
    var count by remember { mutableStateOf(0) }
```

```
    Column(
```

```
        horizontalAlignment = Alignment.CenterHorizontally
```

```
    ) {
```

```
        // (2) TEXT DISPLAYING STATE
```

```
        Text(
```

```
            text = "Button clicked $count times"
```

```
        )
```

```
        Spacer(modifier = Modifier.height(16.dp))
```

```
        // (3) BUTTON DEFINITION
```

```
        Button(
```

```
            onClick = {
```

```
// (4) CLICK HANDLER – STATE MODIFICATION

    count++

    }

    ) {

        Text("Click Me")

    }

}

}
```

Code Annotation & Analysis

✅ Where the Button is Defined:

```
Button(

    onClick = { count++ }

)
```

This is the **Button composable**. It defines the interactive UI element.

✅ Where the Click Handler Modifies State:

```
onClick = {

    count++

}
```

- When the button is tapped,
 - count increases by 1,
 - This updates the mutable state.
-

✅ Which Parts of UI Recompose:

When count changes:

- The Text composable using count recomposes.

- Compose re-runs ClickCounter() and updates only the affected UI.
 - The Button itself does not visually change unless it depends on count.
-

How Compose Handles Recomposition (3–4 Lines)

In Jetpack Compose, UI automatically updates when state changes. When the button is clicked, the count variable (a mutableStateOf) changes. Compose detects this state change and triggers recomposition of any composables that read count. Instead of redrawing the whole screen, Compose efficiently updates only the affected UI elements.

Reflection — Challenges in Designing Interactive UI

1. Managing State Properly

It can be confusing to decide where state should live (local vs ViewModel).

→ I will practice state hoisting and follow recommended architecture patterns.

2. Avoiding Unnecessary Recompositions

Improper state usage can cause performance issues.

→ I will learn how recomposition works internally and use tools like Layout Inspector.

3. Handling Complex User Interactions

Multiple buttons, toggles, and conditions can make logic messy.

→ I will break UI into smaller composables and follow clean code principles.
